

Approximation of Loop Subdivision Surfaces for Fast Rendering

Guiqing Li, Canjiang Ren, Jiahua Zhang, and Weiyin Ma

Abstract—This paper describes an approach to the approximation of Loop subdivision surfaces for real-time rendering. The approach consists of two phases, which separately construct the approximation geometry and the normal field of a subdivision surface. It first exploits quartic triangular Bézier patches to approximate the geometry of the subdivision surface by interpolating a grid of sampled points. To remedy the artifact of discontinuity of normal fields between adjacent patches, a continuous normal field is then reconstructed by approximating the tangent vector fields of the subdivision surfaces with quartic triangular Bézier patches. For regular triangles, the approach reproduces the associated subdivision patches, quartic three-directional box splines.

Index Terms—Subdivision surfaces, graphics processors (GPU), Bézier patches, tessellation, surface approximation.

1 INTRODUCTION

FAST rendering of subdivision surfaces has been investigated by many researchers due to their wide use in 3D modeling systems. Generally, subdivision surfaces are generated through mesh refinement level by level. From the view of rendering, subdivision surfaces may be obtained by procedurally subdividing the control meshes in either breadth-first or depth-first ways [1]. One can also employ an analytic form by Stam to exactly evaluate points on a subdivision surface [2]. Unfortunately, most of the subdivision schemes do not admit analytic form. In addition, exact evaluation is not always effective especially for sharp feature modeling. Approximating subdivision surfaces using parametric surfaces is then a good substitute for visual rendering [3], [4], [5].

Hardware acceleration of subdivision surfaces can be dated back to Pulli and Segal's work for Loop subdivision surfaces [6]. They packed two triangles sharing an edge together to form a square surface patch. The hardware architecture of Bischoff et al. [1] evaluates surface points using forward difference. The approach only works for subdivision surfaces having analytic representation. Bolz and Schröder extended the approach to arbitrary subdivision surfaces by numerically computing the basis functions and stored them in a lookup table [7]. The resolution of the refined meshes is limited by the size of the lookup table in their

approach. Yasui and Kanai realized a high-quality assessment technique for interrogating Catmull-Clark subdivision surfaces based on GPU [8] by casting Stam's evaluation approach [2] onto the accelerated mechanism developed in their early work [9]. Though these approaches are very effective for control meshes with regular valence, however, meshes with large valences in practical applications may leave less memory to other information that is important for rendering, such as texture maps and SVBRDFs [10].

PiPGH preserves adjacency information by maintaining a set of domain atlas for encoding the boundary information between refined face patches in GPU memory [11]. Considering that face-based rendering generally requires extra storage in addition to 2D texture buffers for dealing with extraordinary vertices, Shiue et al. [12] addressed the issue by introducing a vertex-based technique. The control mesh should be subdivided once in CPU in order to obtain refinable fragment meshes for Catmull-Clark subdivision. The method is not applicable to models with arbitrary valence as it needs to tabulate subdivision stencils for different valences in advance. Moreover, the situation is rather complex for Loop subdivision as fragment meshes of control vertices cannot form a watertight surface after a round of subdivision [13].

Vlachos et al. introduced PN-triangles to improve visual quality of 3D models [14]. The approach uses a PN-triangle, which consists of a cubic triangular Bézier patch and a quadratic triangular Bézier normal patch, to replace a triangle of a given mesh. Chung and Kim [15] discussed a specific hardware for rendering PN-triangles. Fünfzig et al. improved the approach with PNG1-triangles [16]. They employed three patches for each triangle to ensure G^1 continuity across edges. A PNG1-triangle is then obtained by blending the three patches. By employing displacement maps to enrich surface details, Boubekur and Schlick proposed a technique for generic mesh refinement on GPU, which is mainly concerned with topological connectivity of mesh refinement instead of the evaluation of subdivision surfaces [17], [18].

Unlike CPU computation, it is usually difficult to use the geometry shader to perform full subdivision with GPU-based processing. In fact, the geometry shader was not designed for large-scale tessellation. Kazakov approximated Catmull-Clark surfaces by a two-level refined mesh whose vertices

- G. Li is with the School of Computer Science and Engineering, South China University of Technology, North District 7-208#, Wushan, Guangzhou 510640, China. E-mail: liqg@scut.edu.cn.
- C. Ren is with the School of Computer Science and Engineering, South China University of Technology, Guangzhou Higher Education Mega Center, Guangzhou 510006, China, and Hong Kong Polytechnic University. E-mail: ren.canjiang@mail.scut.edu.cn.
- J. Zhang is with the Department of Computing, Hong Kong Polytechnic University, Hung Hom, Kowloon, Hong Kong. E-mail: newzjh@126.com.
- W. Ma is with the Department of Manufacturing Engineering and Engineering Management, City University of Hong Kong, 83 Tat Chee Avenue, Kowloon, Hong Kong. E-mail: mewma@cityu.edu.hk.

Manuscript received 25 Dec. 2008; revised 23 May 2009; accepted 27 Oct. 2009; published online 26 May 2009.

Recommended for acceptance by J. Stam.

For information on obtaining reprints of this article, please send e-mail to: tcvg@computer.org, and reference IEEECS Log Number TVCG-2008-12-0209. Digital Object Identifier no. 10.1109/TVCG.2010.83.

are explicitly given [19]. The method does not support further refinement and is also hard to be extended to other schemes. Boubekur and Schlick introduced a so-called quadratic approximation surface (QAS) [3], [4], which uses quadratic Bézier patches to approximate the geometry and normal of a Loop subdivision patch associated with a control triangle, respectively. Geometric and normal patches are constructed by interpolating limit attributes of six points on the triangle associated with the first round of subdivision. Phong tessellation may reduce contour artifacts but it simultaneously suppresses the quality of interior areas [20]. Alexa and Boubekur [21] further applied the idea of Phong shading to reduce visual artifacts for many subdivision schemes. Loop and Schaefer [5] approximated the geometry and tangent vector fields of a Catmull-Clark subdivision surface using a set of bicubic Bézier patches and a set of 2×3 Bézier patches, respectively. The approach aims at applications for the next generation of GPUs with tessellation units—tessellators [22]. Kovacs et al. improved the approach for surfaces with crease features [23].

In the literature, a lot of methods have been invented to skin arbitrary triangular meshes using smoothly stitched parametric surfaces. Some of them also addressed the approximation of subdivision surfaces [24], [25]. However, these methods generally need to refine control meshes at least once for separating extraordinary vertices.

Motivated by the aforementioned approaches separately addressing geometry approximation and normal continuity for fast rendering, this paper addresses the approximation of Loop subdivision surfaces [26] for fast rendering on GPUs with tessellators.

We construct Bézier patches using an interpolating method. Compared with the QAS method [4], our approach can exactly reproduce subdivision patches associated with regular triangles and approximate irregular triangles with smaller errors. The approach is similar to the algorithm of Loop and Schaefer [5].

Different from tensor product patches, however, a triangular Bézier surface has three tangent patches. It is necessary not only to guarantee smooth transition of the normal field when crossing the boundary, but also to maintain the compatibility among the three tangent patches for each triangle. Noticing that there are three combinations, and therefore, three normal fields for three tangent patches, we address the problem by carefully selecting sampled data for the construction of the tangent patches followed by a sophisticated blending of the three normal fields. As a simpler alternative, the normal field can also be reconstructed by directly interpolating the sampled normals using triangular Bézier patches.

The proposed method is also quite flexible. It provides explicit formulas for computing control coefficients of Bézier patches for common triangles. On the other hand, sharp feature treatment is more efficient by multiplying local subdivision matrices. The approach can be implemented in the current graphics pipelines by instantiating tessellation and should be able to benefit from and be more efficient for the next generation GPU [27].

2 PRELIMINARIES

We first introduce geometric rules and masks for evaluating the limit position and tangent vectors along adjacent edges

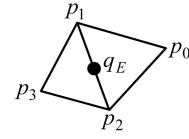


Fig. 1. Vertex labeling for an edge vertex.

for a control vertex of Loop subdivision. A control vertex has different positions in different subdivision levels. Their relationship is recalled. Finally, we list the expression for triangular Bézier patches for unifying formulation.

2.1 Loop Subdivision

Loop subdivision [26] is defined on a triangular control mesh and creates a global C^2 continuous surface except at limit positions of extraordinary control vertices at which the surface is C^1 continuous. Hoppe et al. added sharp features to the scheme [28].

Subdivision masks. Loop subdivision has two kinds of geometric rules. Given a vertex p_0 and its 1-ring adjacent vertices $\{p_i, i = 1, \dots, n\}$, it is modified to

$$p_0^1 = (1 - \alpha_n)p_0 + \frac{\alpha_n}{n} \sum_{i=1}^n p_i, \quad (1)$$

where $\alpha_n = \frac{5}{8} - (\frac{3}{8} + \frac{2}{8} \cos \frac{2\pi}{n})^2$ in [26]. On the other hand, for a given edge (p_1, p_2) , a newly inserted edge vertex can be defined as

$$q_E = \frac{1}{8}(p_0 + 3p_1 + 3p_2 + p_3), \quad (2)$$

where p_0, p_3 are opposite vertices of the edge as shown in Fig. 1.

Limit masks. The limit position of p_0 can be computed using the same stencil as that for computing the updated control vertex but with different weights (see [28], for example):

$$p_0^\infty = \frac{\omega_n}{n + \omega_n} p_0 + \frac{1}{n + \omega_n} \sum_{i=1}^n p_i, \quad (3)$$

where $\omega_n = \frac{3n}{8\alpha_n}$.

From (1) and (3), we can also derive the relationship of p_0 , p_0^1 , and p_0^∞ by canceling the sum of incident vertices. Generally, we can show [29]

$$p_0^{l+1} = \left(\frac{5}{8} - \alpha_n\right)p_0^l + \left(\frac{3}{8} + \alpha_n\right)p_0^\infty. \quad (4)$$

Tangent masks. Consider the subdivision matrix for a 1-ring stencil of valence n :

$$M = \begin{pmatrix} 1 - \alpha_n & \alpha_n & \alpha_n & \cdots & \alpha_n \\ \frac{\alpha_n}{n + \omega_n} & \frac{\alpha_n}{n + \omega_n} & \frac{\alpha_n}{n + \omega_n} & \cdots & \frac{\alpha_n}{n + \omega_n} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ \frac{\alpha_n}{n + \omega_n} & \frac{1}{8} & 0 & \cdots & \frac{\alpha_n}{n + \omega_n} \end{pmatrix}.$$

Its two subdominant eigenvalues $\lambda_{1,2} = \frac{1}{8}(3 + 2 \cos \frac{2\pi}{n})$ are associated with the following two eigenvectors [28]:

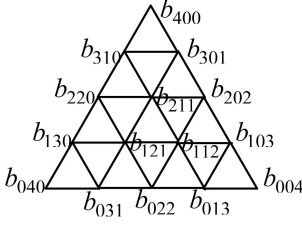


Fig. 2. Organization of control points of a quartic triangular Bézier surface.

$$\alpha_1 = \left[0, \left(\cos \frac{2\pi i}{n} \right)_0^{n-1} \right]^T \text{ and } \alpha_2 = \left[0, \left(\sin \frac{2\pi i}{n} \right)_0^{n-1} \right]^T,$$

where superscript T indicates transpose and notation $(f(i))_0^\eta$ stands for vector $f(0), f(1), \dots, f(\eta)$. This yields control vertices $\{(0,0)\} \cup \{(\cos \frac{2\pi i}{n}, \sin \frac{2\pi i}{n})\}_{i=0}^{n-1}$ of the characteristic map. Similar to [5], we also apply the tangent stencil to the control net of the characteristic map and further scale the corresponding tangent vector to $(1,0)$ using γ :

$$\gamma \sum_{0 \leq i \leq n} \cos \frac{2\pi i}{n} \left(\cos \frac{2\pi i}{n}, \sin \frac{2\pi i}{n} \right) = (1,0).$$

It follows that $\gamma = \frac{2}{n}$, which finally leads to the tangent weights

$$\alpha = \gamma \alpha_1 = \left[0, \left(\frac{2}{n} \cos \frac{2\pi i}{n} \right)_0^{n-1} \right]^T. \quad (5)$$

Namely, the tangent vector at p_0 along edge $\overrightarrow{p_0 p_i}$ is

$$\mathbf{t}_i = \sum_{j=0}^{n-1} \alpha[j] p_{1+\text{mod}(i+j-1,n)}, \quad i = 1, \dots, n, \quad (6)$$

where $\alpha[j]$ stands for the j th entry of α . It is easy to show that these tangent vectors satisfy

$$\mathbf{t}_{\text{mod}(n-2+i,n)+1} + \mathbf{t}_{\text{mod}(i,n)+1} = \mathbf{t}_i 2 \cos \frac{2\pi}{n}, \quad i = 1, \dots, n. \quad (7)$$

2.2 Triangular Bézier Patches

A triangular Bézier patch of degree m is defined as the blending of $\frac{1}{2}(m+1)(m+2)$ control points:

$$B_m(r, s, t) = \sum_{i+j+k=m} \mathbf{b}_{ijk} \frac{m!}{i!j!k!} r^i s^j t^k, \quad r+s+t=1. \quad (8)$$

For discrimination from control vertices of subdivision surfaces, we name the control points of $B_m(r, s, t)$ as control coefficients or simply coefficients.

3 APPROXIMATION OF SUBDIVISION SURFACE GEOMETRY

As Loop subdivision generalizes three-directional box splines of degree 4, we employ a quartic triangular Bézier patch to approximate the Loop subdivision patch of a control triangle. Instead of using basis conversion to transform a box spline patch into a triangular Bézier patch [30], [31], we directly attack the problem by interpolation.

3.1 Approximation by Interpolating Bézier Patches

Let τ be a triangle on the given control mesh and $F_\tau(r, s, t)$ the subdivision patch determined by the 1-ring stencil of τ . We want to construct a $B_4(r, s, t)$ interpolating the following points of $F_\tau(r, s, t)$:

$$B_4(r_i, s_j, t_k) = F_\tau(r_i, s_j, t_k) \quad \text{for } r_i = 2^{i-4}, s_j = 2^{j-4}, \\ t_k = 1 - r_i - s_j, \quad (0 \leq i \leq 4, 0 \leq j \leq 4-i, k = 1-i-j), \quad (9)$$

Denote by $\mathbf{B}$ the column vector with entries $\mathbf{B}[(4-i)(4-i+1)/2+k] = \mathbf{b}_{ijk}$, which numerates the 15 coefficients of $B_4(r, s, t)$ from left to right and then from top to bottom as shown in Fig. 2. Let $\mathbf{F}$ be a 15-dimensional column vector with entries $\mathbf{F}[(4-i)(4-i+1)+k] = F_\tau(r_i, s_j, t_k)$. In addition, introduce matrix $\mathbf{M}_B$ such that

$$\mathbf{M}_B[(4-i)(4-i+1)+k, (4-i')(4-i'+1)+k'] \\ = \frac{4!}{i'!j'!k'!} r_i^{i'} s_j^{j'} t_k^{k'}.$$

Equation (9) can be written as the matrix form

$$\mathbf{M}_B \mathbf{B} = \mathbf{F}. \quad (10)$$

Generally, points $F_\tau(r_i, s_j, t_k)$ can be obtained by subdividing the 1-ring stencil of τ twice and then computing the limit position of the vertices on τ , or evaluated directly in parametric form [31]. The control coefficients of $B_4(r, s, t)$ can be further found by solving (10). To make the computation more efficient, we perform subdivision rules on the local submesh to derive a compact representation.

3.2 Construction of Bézier Patches

Let $\tau = (p_0, p_1, p_2)$ (gray area in Fig. 3a) and n_i be the valence of p_i ($i = 0, 1, 2$). We first define a **work template** for τ based on its 1-ring neighborhood. The template consists of 16 triangles: τ itself in addition to five for each of its three edges. For edge $p_i p_{i+1}$, the five triangles include the opposite one τ' of τ in terms of p_i , two triangles clockwise succeeding τ' in the 1-ring of p_i , and two triangles counterclockwise succeeding τ' in the 1-ring of p_{i+1} . We relabel the template vertices using indices 0-17 as depicted in Fig. 3a. Note that vertices in the control mesh may repeat in the template for $n_i \leq 7$. For instance, we have $p_3 = p_{16} = p_1$, $p_4 = p_{17} = p_2$, and $p_5 = p_{15}$ for $n_0 = 3$.

Let q_i^∞ ($i = 0, 1, 2$) be the limit position of p_i defined by (3), and $\mathbf{P} = [(p_i)_0^{17}, q_0^\infty, q_1^\infty, q_2^\infty]^T$, where q_i^∞ are used to evaluate the new position of the three vertices of τ after the first round of subdivision. For convenience, we also name the template of τ as $\mathbf{P}$.

Next, we subdivide $\mathbf{P}$ and extract a submesh $\mathbf{P}_1$ of τ from the refined stencil (region within broken lines in Fig. 3b). Numbers in Fig. 3b show the labeling system for vertices of $\mathbf{P}_1$. Denote $\mathbf{P}_1 = [(p_i^{(1)})_0^{17}, q_0^\infty, q_1^\infty, q_2^\infty]^T$, where the parenthesized superscript (1) indicates the subdivision level. All vertices of $\mathbf{P}_1$ can be evaluated using the geometric rule of (2) except for $p_0^{(1)}$, $p_3^{(1)}$, and $p_5^{(1)}$, which correspond to three corners of τ and can be computed using (4). It results in

$$\mathbf{P}_1 = \mathbf{M}_1 \mathbf{P}, \quad (11)$$

where $\mathbf{M}_1$ is a 21×21 matrix whose nonzero entries are listed in Table 1 of Appendix A.

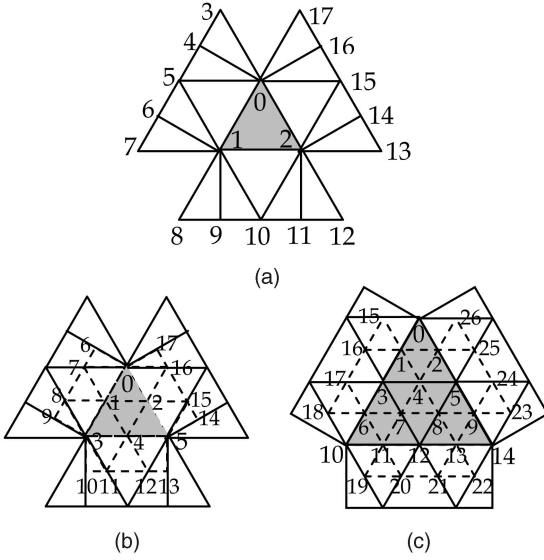


Fig. 3. Template evolution for evaluating geometric attributes of 15 uniformly resampled points on τ (the gray triangular area): (a) the work template of τ , (b) and (c) submeshes (dotted line area) after one and two rounds of subdivision, respectively.

Similarly, perform Loop subdivision on stencil $\mathbf{P}_1$ and extract a refined submesh $\mathbf{P}_2$ (region within broken lines in Fig. 3c), which provides sufficient information for calculating position and tangent of vertices on τ . Again, Fig. 3c gives numeration of the vertices on submesh $\mathbf{P}_2$. Denote $\mathbf{P}_2 = [(p_i^{(2)})_0^{26}, q_0^\infty, q_1^\infty, q_2^\infty]^T$. By combining (2) and (4), we obtain

$$\mathbf{P}_2 = \mathbf{M}_2 \mathbf{P}_1, \quad (12)$$

where $\mathbf{M}_2$ is a 30×21 matrix (see Table 2 of Appendix A for details).

So far we have created all information for evaluating the limit position and tangent vectors of 15 vertices on τ generated after two steps of subdivision. Let $\mathbf{P}_\infty^T = (p_i^\infty)_0^{14}$ be the vector of limit positions of $\{p_i^{(2)}\}_0^{14}$. By applying the limit position rule of (3), we obtain

$$\mathbf{P}_\infty = \mathbf{M}_\infty \mathbf{P}_2, \quad (13)$$

where $\mathbf{M}_\infty$ is a 15×30 matrix (see Table 3 of Appendix A). Substituting $\mathbf{P}_\infty$ for $\mathbf{F}$ in (10) yields the following equation:

$$\mathbf{B} = \mathbf{M}_B^{-1} \mathbf{P}_\infty = \mathbf{M}_B^{-1} \mathbf{M}_\infty \mathbf{M}_2 \mathbf{M}_1 \mathbf{P}. \quad (14)$$

In (14), $\mathbf{M}_B^{-1}$ and $\mathbf{M}_\infty$ are two constant matrices while most entries of $\mathbf{M}_2$ and $\mathbf{M}_1$ are constant except for a few which are a function of three valences. See Appendix B for the closed form of coefficients in $\mathbf{B}$.

As the triangular Bézier surface defined by coefficient vector $\mathbf{B}$ interpolates 15 points of the subdivision patch of τ , it reproduces the patch if the three vertices of τ are regular (valence 6).

4 APPROXIMATION OF TANGENT VECTOR FIELDS

Bézier patches established in Section 3 form a C^0 approximation of the underlying subdivision surface. Here, we also establish a continuous normal field by reconstructing

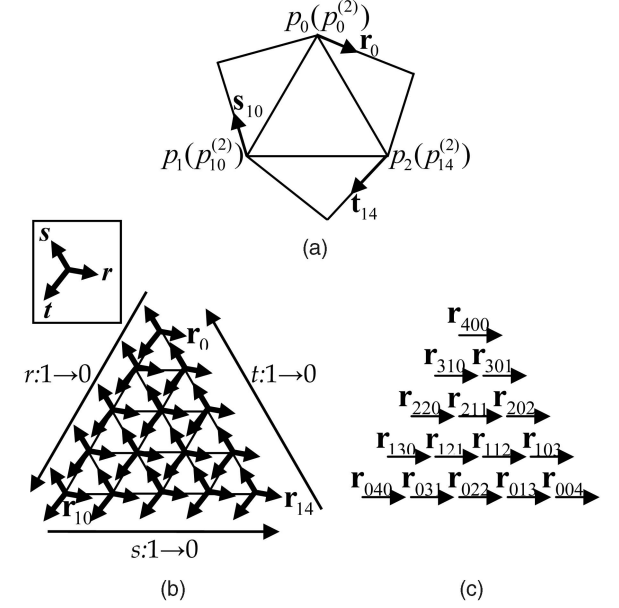


Fig. 4. Tangent fields: (a) selection of $\mathbf{r}_0$, $\mathbf{s}_{10}$, and $\mathbf{t}_{14}$; (b) three discrete tangent fields along r , s , and t parametric directions for the subdivision patch of τ ; and (c) layout of the control stencil for $R_4(r, s, t)$. Those of $S_4(r, s, t)$ and $T_4(r, s, t)$ have the same index order.

tangent vector fields in order to achieve more realistic rendering.

In the setting of Loop and Schaefer's approach [5], there are one tangent vector curve of degree 3 and two tangent vector curves of degree 2 along the common boundary of two adjacent bicubic Bézier patches. This makes it possible to force the three vector curves linearly dependent via modifying the coefficients of the tangent patches. However, directional derivatives of triangular Bézier patches of degree n reduce to a canonical triangular Bézier patch of degree $n - 1$. The method does not work in our case.

Note that $\mathbf{P}_2$ obtained in Section 3 is sufficient for estimating tangent vectors of vertices inserted into τ . Assume that in the original mesh, tangent vectors of each vertex along all edge directions have been computed using (6). In the following, we demonstrate how to establish the discrete tangent fields along r , s , and t isoparametric edges, respectively.

Denote $\mathbf{R} = [(\mathbf{r}_i)_0^{14}]^T$, where $\mathbf{r}_i$ except for $\mathbf{r}_0$ are uniquely determined as the r tangent vector at the limit position of $p_i^{(2)}$ while $\mathbf{r}_0$ is undefined. We enforce it to be the tangent vector at p_0 (or equivalently $p_0^{(2)}$) along the edge counter-clockwise next to $p_0 p_2 (p_0^{(2)} p_{14}^{(2)})$ on the subdivision surface (see Fig. 4a). The definition of $\mathbf{r}_0$ is key for guaranteeing boundary continuity. Similarly, let $\mathbf{T} = [(\mathbf{t}_i)_0^{14}]^T$ and $\mathbf{S} = [(\mathbf{s}_i)_0^{14}]^T$ be tangent vector fields for t and s , respectively.

Applying the tangent mask in (6) to Fig. 3c results in

$$\mathbf{R} = \mathbf{M}_R \mathbf{P}_2, \quad (15)$$

where $\mathbf{M}_R$ is a 15×30 matrix whose nonzero entries are tabulated in Table 4 of Appendix A. Note that $\mathbf{r}_0$, $\mathbf{r}_{10}$ and $\mathbf{r}_{14}$ tangent vectors of three vertices of τ have been precomputed. They are still listed in $\mathbf{R} = (\mathbf{r}_i)_0^{14}$ for convenience of description. Correspondingly, we can simply set rows 1, 11, and 15 of $\mathbf{M}_R$ to zero. It then yields the coefficients of a quartic triangular Bézier patch interpolating $\mathbf{R}$:

$$\mathbf{B}_R = \mathbf{M}_B^{-1} \mathbf{R} = \mathbf{M}_B^{-1} \mathbf{M}_R \mathbf{M}_2 \mathbf{M}_1 \mathbf{P}, \quad (16)$$

where $\mathbf{B}_R$ enumerates the coefficients $\mathbf{r}_{ijk}$ in the same order as $\mathbf{B}$ (see Fig. 4c). Finally, we obtain an approximation of the tangent patch along r isoparametric direction of the subdivision patch of τ :

$$R_4(r, s, t) = \sum_{i+j+k=4} \mathbf{r}_{ijk} \frac{4!}{i!j!k!} r^i s^j t^k. \quad (17)$$

The closed forms for $\mathbf{r}_{ijk}$ are listed in Appendix C.

The discrete t tangent vector field on τ can be constructed analogously:

$$\mathbf{T} = \mathbf{M}_T \mathbf{P}_2, \quad (18)$$

where $\mathbf{M}_T$ is constructed in the same way as $\mathbf{M}_R$. Write the triangular Bézier patch interpolating $\mathbf{T}$ as

$$T_4(r, s, t) = \sum_{i+j+k=4} \mathbf{t}_{ijk} \frac{4!}{i!j!k!} r^i s^j t^k, \quad (19)$$

where the control coefficient vector is defined by

$$\mathbf{B}_T = \mathbf{M}_B^{-1} \mathbf{T} = \mathbf{M}_B^{-1} \mathbf{M}_T \mathbf{M}_2 \mathbf{M}_1 \mathbf{P}. \quad (20)$$

We do not establish $S_4(r, s, t)$ in the same way but use a simpler trick to do this. Noticing (7), we have

$$\mathbf{s}_i = -(\mathbf{r}_i + \mathbf{t}_i), 0 \leq i \leq 14, i \neq 0, 10, 14, \quad (21)$$

and

$$\begin{aligned} 2 \cos \frac{2\pi}{n_0} \mathbf{s}_0 &= -(\mathbf{r}_0 + \mathbf{t}_0), \quad 2 \cos \frac{2\pi}{n_1} \mathbf{t}_{10} = -(\mathbf{r}_{10} + \mathbf{s}_{10}), \\ 2 \cos \frac{2\pi}{n_2} \mathbf{r}_{14} &= -(\mathbf{s}_{14} + \mathbf{t}_{14}), \end{aligned} \quad (22)$$

It follows that

$$\mathbf{S} = (\mathbf{s}_i)_{i=0}^{14} = -\frac{1}{2}(\mathbf{R} + \mathbf{T}) + \mathbf{\Pi},$$

where $\mathbf{\Pi}$ is a 15-entry vector with

$$\begin{aligned} \mathbf{\Pi}[0] &= \left(1 - 2 \cos \frac{2\pi}{n_0}\right) \mathbf{s}_0, \quad \mathbf{\Pi}[10] = \left(1 - 2 \cos \frac{2\pi}{n_0}\right) \mathbf{t}_{10}, \\ \mathbf{\Pi}[14] &= \left(1 - 2 \cos \frac{2\pi}{n_0}\right) \mathbf{r}_{14} \quad \text{and} \quad \mathbf{\Pi}[i] = 0, \quad i \neq 0, 10, 14. \end{aligned}$$

This finally results in the following equation:

$$\begin{aligned} S_4(r, s, t) &= -\frac{1}{2}(R_4(r, s, t) + T_4(r, s, t)) \\ &\quad + \mathbf{\Pi}[0]r^4 + \mathbf{\Pi}[10]s^4 + \mathbf{\Pi}[14]t^4, \end{aligned} \quad (23)$$

which greatly simplifies the computation of $S_4(r, s, t)$. $R_4(r, s, t)$, $S_4(r, s, t)$, and $T_4(r, s, t)$ are coplanar for regular τ due to the fact that $\mathbf{\Pi} = \mathbf{0}$ in this case.

Remark 1. Though we employ quartic instead of cubic patches to approximate the tangent fields, the above fields reproduce the tangent fields of the subdivision patch for regular τ . They are actually the degree-elevated tangent fields of the subdivision patch. See Appendix D for the proof of this remark.

Now we analyze the continuity of the normal patches, which are computed using these tangent patches, across a

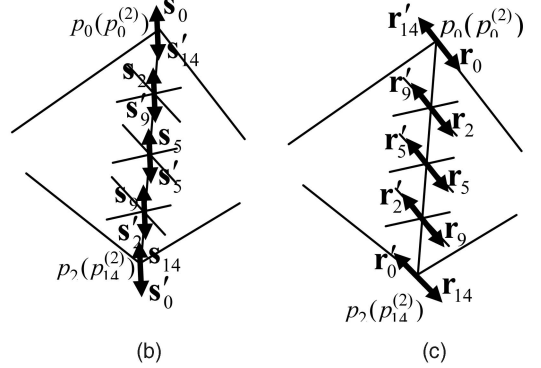
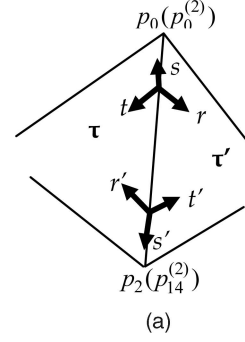


Fig. 5. Tangent vector orientation: (a) orientation of tangent patches of two triangles; (b) control coefficients of s and s' tangent curves; and (c) control coefficients of r and r' tangent curves.

boundary. Suppose that triangle τ' shares p_0p_2 with τ . Without loss of generality, assume that the orientations of their tangent patches along p_0p_2 are arranged as shown in Fig. 5a. It follows that $S_4(r, s, 0) = -S'_4(r, s, 0)$ ($r' = 1 - r$, $s' = 1 - s$) due to the reverse orientations of the parametric directions of the two patches (see Figs. 5a and 5b). Specifically, $\mathbf{s}_0 = -\mathbf{s}'_{14}$ and $\mathbf{s}_{14} = -\mathbf{s}'_0$ are guaranteed by our selection while $\mathbf{s}_2 = -\mathbf{s}'_9$, $\mathbf{s}_5 = -\mathbf{s}'_5$, and $\mathbf{s}_9 = -\mathbf{s}'_2$ hold due to the regularity of the corresponding points. It is also true that $R_4(r, s, 0) = -R'_4(r, s, 0)$ ($r' = 1 - r$, $s' = 1 - s$) for the same reason (see Fig. 4c). This leads to the following:

$$N_{RS}(r, s, 0) = N_{R'S'}(r, s, 0),$$

where $N_{XY}(r, s, t)$ stands for $X_4(r, s, t) \times Y_4(r, s, t)$.

Similarly, $N_{ST}(r, s, t)$ and $N_{TR}(r, s, t)$ are compatible with their adjacent patches along edges p_0p_1 and p_1p_2 , respectively. However, none of them can guarantee that the normal field is continuous along the three boundaries as the three tangent vector patches are generally not compatible for irregular τ due to (18). We remedy the artifact by simply blending three normal patches using normalized weights as follows:

$$\begin{aligned} N(r, s, t) &= \frac{1}{rs + st + tr} [rsN_{RS}(r, s, t) \\ &\quad + stN_{ST}(r, s, t) + rtN_{TR}(r, s, t)]. \end{aligned} \quad (24)$$

Remark 2. To avoid computing (24), an alternative way is to evaluate the normal of vertices 0-14 on the work template in Fig. 3c and then construct a quartic normal Bézier patch passing through these normals. As the normal of a quartic Bézier patch is a six-degree polynomial, it is impossible to

reproduce the normal field for a regular control triangle by this normal interpolation method. Given vertex normals, constructing a smooth normal field via interpolation may date back to the sophisticated method by Overveld and Wyvill [32]. The PN-triangle follows the same idea [14]. The strategy was extended to approximate the normal field of subdivision surfaces by Boubekur and Schlick [4] and Alexa and Boubekur [21].

5 GPU IMPLEMENTATION

We first sketch a one-pass pipeline for implementing our method on GPUs with tessellation feature and then map the pipeline to a two-pass program running on current GPUs. The issue for sharp or semisharp features is also addressed in the section.

5.1 Pipeline for Implementation in DirectX 11

Similar to the framework for approximating Catmull-Clark subdivision [33], [34], our approximation of Loop subdivision can be implemented within the DirectX 11 pipeline in a single pass as follows:

- **Hull shader:** This stage computes control coefficients of approximate Bézier patches and control vectors of tangent Bézier patches.
- **Tessellator:** Finish a configurable tessellation according to a subdivision depth factor.
- **Domain shader:** It evaluates the position and normal of sampled points for a given Bézier patch and the corresponding tangent patches.

Specifically, our hull shader has the form as shown in Listing 1 which is rewritten from the hull shader of the “SubD11” example in DirectX SDK. Its inputs include the index of the patch and the index set of control vertices of the patch. It first loads the precomputed data of the patch from buffers and then selects masks to calculate the control coefficients according to their indices (see Appendix B) for geometric and tangent Bézier patches. The hull shader deals with one control coefficient in each time, therefore, it is invoked 15 times in order to construct a Bézier patch.

```
BÉZIER_CONTROL_POINT SubDToBézierHS (
InputPatch<VS_CONTROL_POINT_OUTPUT, MAX_POINTS> p,
uint i : SV_OutputControlPointID, uint PatchID : SV_PrimitiveID)
{
    Buffer.Load(SV_PrimitiveID); // loaded patch data from a buffer
    switch(i) {
        case 5,6,10,9:
            Output.vPosition = interior mask of Appendix B from (14);
            break;
        case 1,2,13,14,4,8,7,11:
            Output.vPosition = edge mask of Appendix B from (14); break;
        case 0,3,15,12:
            Output.vPosition = vertex mask from (14); break;
    }
    Output.rTangent = tangential patch from (16);
    Output.tBinormal = tangential patch from (20);
    Output.sBinormal = tangential patch from (21) and (22);
    return Output;
}
```

Listing 1: Hull shader.

Listing 2 presents the domain shader, which is similar to the “BézierEvalDS” function in the “SubD11” example. It first evaluates the Bernstein bases for given parametric values and then employs the Bézier form (8) and the blending (24) to work out the point positions and their normals, respectively.

```
[domain("quad")]
DS_OUTPUT BézierEvalDS( HS_CONSTANT_DATA_OUTPUT
input, float2 uv : SV_DomainLocation,
const OutputPatch<BÉZIER_CONTROL_POINT, 15> bezpatch )
{
    float4 BasisUVW = BernsteinBasis(u,v,1-u-v);
    float3 pworld = evaluate using (8);
    Output.vPosition = mul( float4(pworld,1), g_mViewProjection );
    Output.vNormal = blending normal from (24);
    return Output;
}
```

Listing 2: Domain shader.

5.2 Simulation on Current GPUs

We first compute control coefficients in $\mathbf{B}$, $\mathbf{B}_R$, $\mathbf{B}_S$, and $\mathbf{B}_T$ for each Bézier patch using a vertex shader. As current GPUs do not fully support hardware tessellation, we have to initiate a new pass to evaluate sampling points on a Bézier patch using a vertex shader.

Concretely, we create a half-edge structure in CPU from which GPU can only read data and to which CPU can write data. It consists of four arrays: vertex, edge, face, and position buffers stored in ID3DX10Buffers for GPU retrieving. Specifically, each cell of the face buffer contains three half-edge indices of the corresponding face; each entry of the edge buffer stores the indices of the vertex emitting this half edge, the next half-edge in the loop of the face, the opposite half-edge, and the face.

Our simulation can be organized as the following two passes based on the above structure:

- **Preprocessing Pass:** In the vertex shader, we first retrieve the patch information (indices of three neighboring patches, and indices of its control vertices) from the half-edge structure buffers through HLSL10 instruction `buffer.Load()`, and then calculate the triangular Bézier control coefficients of (14) using the closed form found in Appendix B, LOD level and visibility. In the geometry shader, we only stream out each visible patch in the form of a dual vertex of the patch. This pass does not involve the pixel shader. The streamed-out patch information is bounded again as input assembly for next pass using “Drawauto” call.
- **Rendering Pass:** Before launching this pass, it needs to read back the information of the stream-out patch by a temporary stage buffer with CPU access permission, and then select a pregenerated instancing pattern like ARP [17], [18]. The vertex shader of the pass simulates the work of the domain shader in Section 5.1. Displacement mapping and geometry texture are optional.

5.3 Sharp Feature Modeling

For open control meshes, one can add a strip of triangles to the boundary such that the original boundary triangles have the same behavior as that of the interior triangles [5]. However, the strategy does not work for modeling sharp or semisharp features. As there are a lot of possible configurations for the stencil in Fig. 3 if sharp features are allowed, it is difficult to exhaust all of them. We suggest constructing the matrices in (14), (16), and (20) according to sharp feature tags based on a modified half-edge structure and then obtaining control coefficients by sparse matrix multiplication instead of deriving analytic expressions.

To construct geometric Bézier approximation, geometric masks for sharp feature modeling introduced by Hoppe et al. [28] are employed to derive M_1 and M_2 in (11) and (12) first (see Fig. 3). If sharp features are desirable, we have to continue to employ limit position masks for all kinds of sharp feature vertices derived in [28] to evaluate M_∞ . Nevertheless, M_∞ is the same as that of the case without sharp features if semisharp features are required (assume that the semisharp feature scheme consists of first two steps of sharp subdivision and the rest steps with smooth subdivision). To address the issue, one needs to further establish the relationship between different level positions for a sharp feature vertex similar to (4).

For the construction of tangential patches, M_1 and M_2 are still reusable. Analogously, M_R and M_T are the same as those for smooth control meshes if semisharp features are considered. However, Hoppe et al. only presented two tangent vectors for determining the tangential plane for a boundary or sharp feature vertex instead of its tangent vectors along all edges. We assign a vector to each edge direction which is sampled on the tangential plane.

6 RESULTS AND DISCUSSION

We abbreviate our approximation approach using quartic triangular Bézier patches by QTB. Furthermore, denote QTB with normal calculated from geometric patches, normal Bézier patches (see Remark 2), and blending normal defined by Peters (24) by QTBG, QTBN, and QTBT, respectively. This section first presents an example to show the visual quality by QTB, and then compares QTB with the state-of-the-art methods, PN-triangles (PNs) [14] and quadratic approximation (QAS) [4] in terms of geometric and normal errors. Here, PN-triangles always interpolate the limit position of control vertices and each of the PN-triangles is associated with a separated quadratic normal patch passing through the limit normals of the three control vertices for rendering. It then gives some results rendered based on GPU. At the end of the section, we present a brief analysis to show that the proposed approach outperforms the smooth patching (SP) by Peters [24] in rendering efficiency.

6.1 Visual Comparison

Fig. 6 presents an example to show the visual quality of the surfaces produced using QTBG (Fig. 6c), QTBN (Fig. 6d), and QTBT (Fig. 6e), respectively. Fig. 6b is the subdivision surface of the control mesh in Fig. 6a. It is easy to observe that approximation of QTBG only achieves C^0 continuity

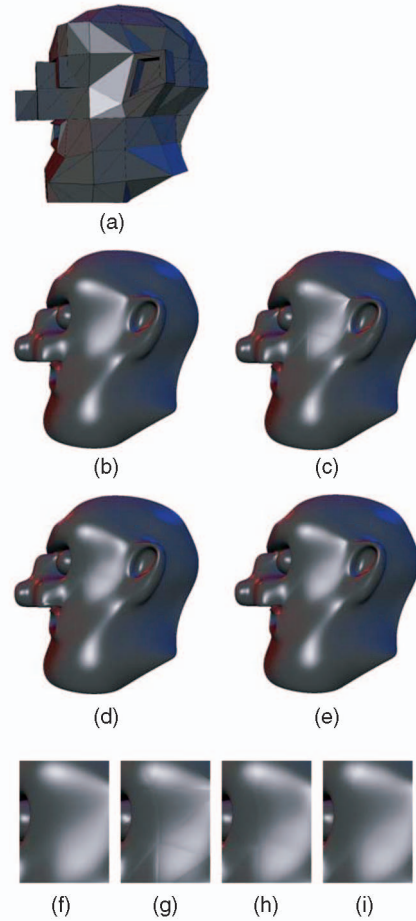


Fig. 6. Rendering example: (a)-(e) control mesh, limit Loop subdivision surface after four steps, QTBG, QTBN, and QTBT surfaces, respectively. (b')-(e') are, respectively, a zoomed region from (b')-(e').

while results by QTBN and QTBT are very close to the subdivision surface.

6.2 Geometric and Normal Errors

QTB exactly reproduces the subdivision patch if the corresponding triangle is regular while PN and QAS cannot. In irregular regions, QTB also exhibits higher accuracy. Fig. 7 illustrates two examples for visually comparing the three techniques. Both QTBN (Fig. 6d) and QTBT (Fig. 7e) are better than PN (Fig. 7b) and QAS (Fig. 7c) in approximating normal fields of the subdivision surfaces.

To intuitively observe the error distribution, Fig. 8 illustrates the geometric error between a limit point on the subdivision surface and the corresponding point on the approximation surface with the same parametric value. In the figure, the geometric error is converted into a relative error, which is defined as the ratio of the absolute geometric error divided by the diagonal length of the bounding box of the entire model. Note that QBTN and QBTT share the same geometric approximation.

Fig. 9 measures the angle between normals of point pair on a subdivision surface and its approximating counterpart. Similarly, QTBN and QTBT produce superior normal field

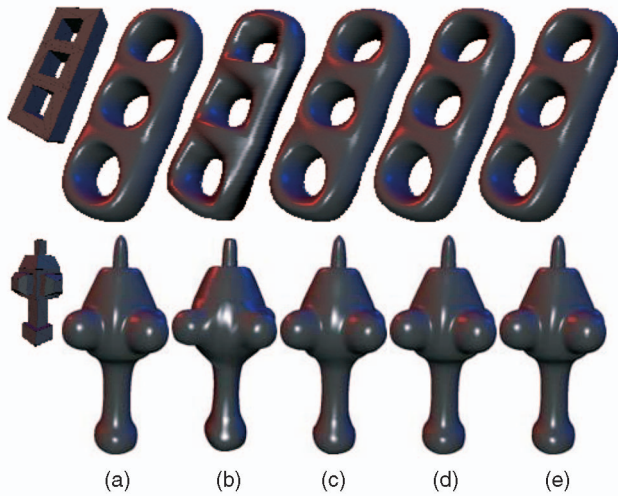


Fig. 7. Visual comparison: (a) subdivision surfaces after four steps, and approximate surfaces using (b) PN, (c) QAS, (d) QTBN, and (e) QTBT, respectively.

than PN and QAS. On the other hand, it can be observed that the normal field by QTBT (the top row of Fig. 9d) is better than the one by QTBN (the top row of Fig. 9c) for the three-hole model while the case is vice versa for the spindle-like model (see the bottom row of Figs. 9c and 9d). Via observation, we find that QTBN may result in a larger normal error if the normal varies drastically within a triangular subdivision patch while QTBT generally leads to distorted normal field for control triangles of bad shape (e.g., triangles with high-valence vertices). According to our experience, however, both approaches produce very similar realistic rendering results for most cases.

Fig. 10 is designed to assess the relationship between the approximation error and the control mesh size of the three approaches. X-axis stands for the vertex number of control meshes (scaled by 100) while Y-axis represents the maximal geometric error (top diagram) and maximal angle error (bottom diagram). The figure demonstrates that QTBT

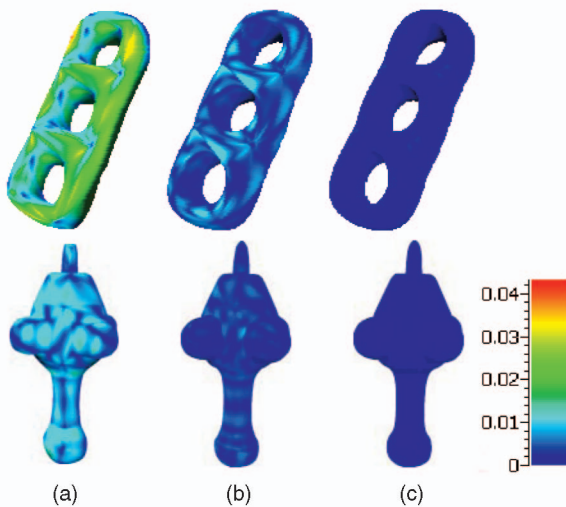


Fig. 8. Geometric error using (a) PN, (b) QAS, and (c) QTBT.

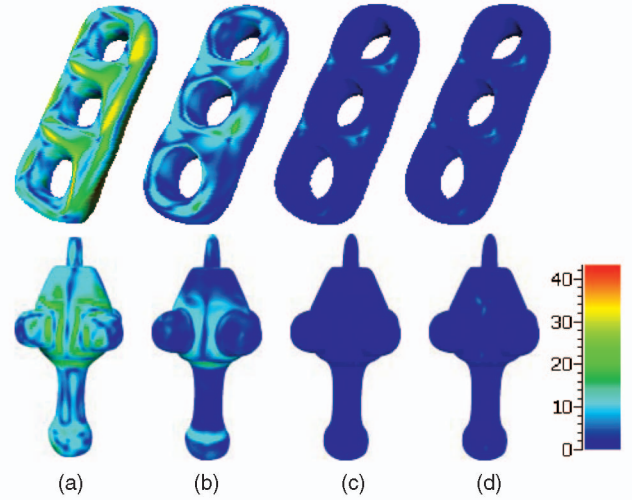


Fig. 9. Normal angle errors using (a) PN, (b) QAS, (c) QTBN, and (d) QTBT.

substantially outperforms the other two methods. We also recorded the processing time for computing coefficients of a Bézier patch and the time for tessellating the Bézier patch, respectively, for all control meshes in this example. The latter generally accounts for about 90 percent of the total time.

Fig. 11 presents the reflection lines of the Loop subdivision surfaces and their Bézier patch approximations. It shows that the latter can faithfully capture the fairness of the corresponding subdivision surfaces.

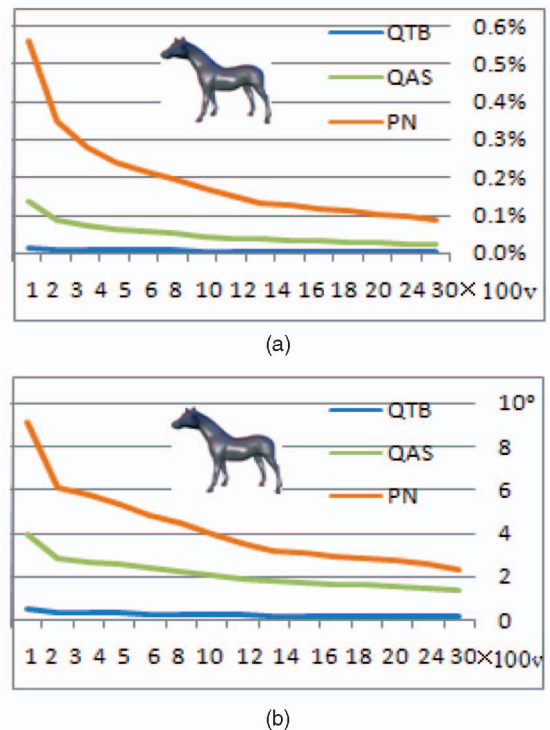


Fig. 10. Relationship between error and input size for a horse model: (a) maximal geometric error and (b) maximal normal error.

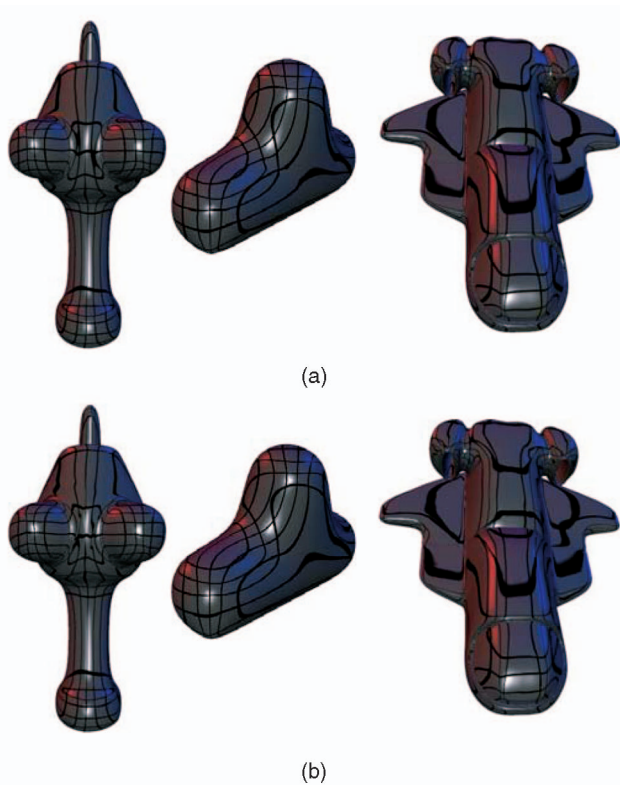


Fig. 11. Reflection lines for three models: (a) the exact subdivision surfaces and (b) drawn with our approximate geometry patches plus normal patches defined by Peters (24).

6.3 Sharp and Semisharp Features

Fig. 12 shows that our approach reproduces the appearance of subdivision surfaces very well. Fig. 13 shows the rendering effect using smooth, semisharp feature, and sharp feature modes, respectively. It demonstrates that subdivision surfaces with semisharp/sharp features look more pleasing in some cases. Furthermore, semisharp features are more

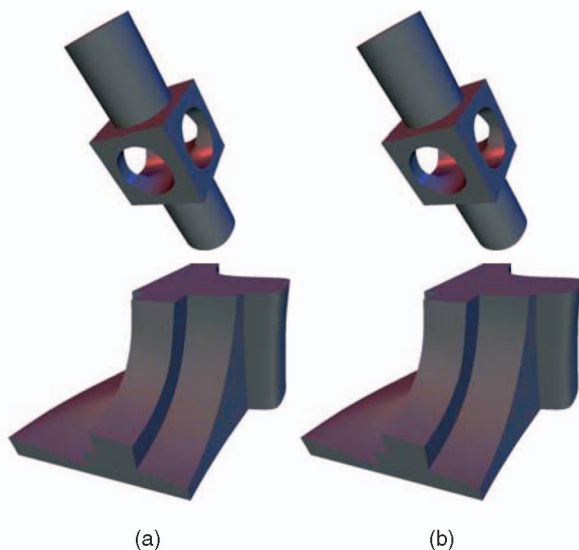


Fig. 12. Sharp feature modeling: (a) and (b) are, respectively, the exact subdivision surfaces and QBT approximations of Mechpart (top) and Fandisk (bottom) models.

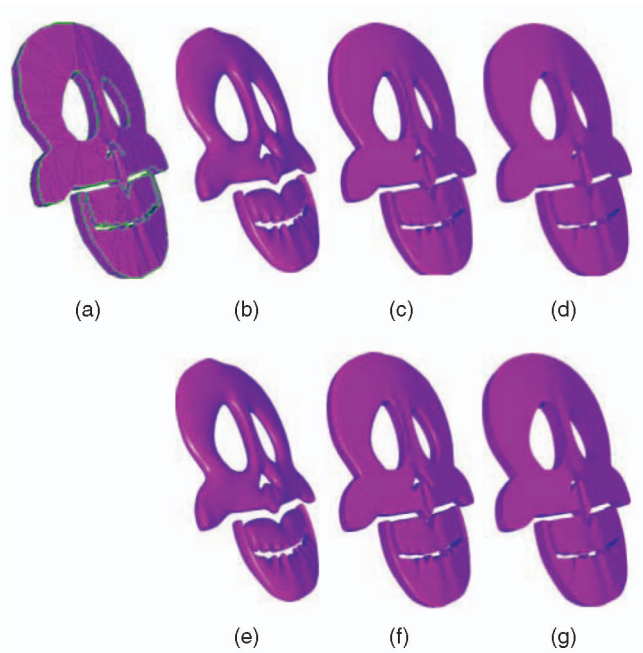


Fig. 13. Sharp feature modeling: (a) the initial control mesh with sharp feature tags (green edges). (a)-(c) The smooth, semisharp feature, and sharp feature surfaces, respectively. Illustrations of the top row are produced with exact subdivision surfaces, while those of the bottom row are produced with QBT approximations.

desirable than sharp features and their modeling with the first two steps of sharp feature subdivision is generally sufficient in practice.

6.4 Examples of GPU Implementation

Our GPU implementation runs on a portable computer with an NVidia 8400M G laptop GPU. Fig. 14 presents two examples generated by GPU. The right two pictures illustrate adaptive tessellated results and the left two pictures are, respectively, their realistic rendering results. Here, the refinement level of each patch is inversely proportional to the distance from the viewpoint to the centroid of the corresponding control triangle. To correctly connect two neighboring regions of different resolutions, we use the snap function by Dyken et al. to remove triangulation cracks [35].

Finally, we present a comparison about GPU rendering efficiency between QAS and our QTB (both QTBN and QTBT are considered here). Only normals of tessellation

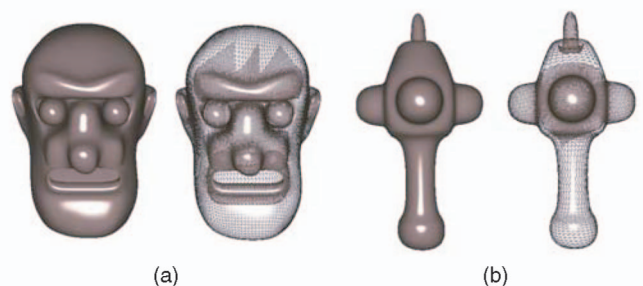


Fig. 14. Adaptive tessellation of Bézier patch approximations: the tessellation level is reversely proportional to the distance from the viewpoint to the centroid of the control triangle.

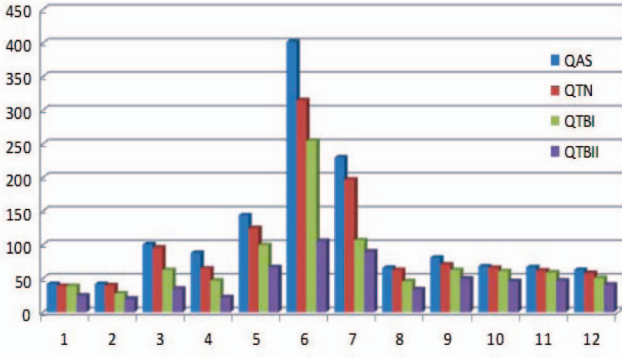


Fig. 15. Comparison of frames per second for QAS, QTBN, QTBT I, and QTBT II. The Y-axis depicts the FPS numbers and there are, in total, 12 groups of data along the X-axis.

vertices are evaluated for QAS and QTBN, while two rendering modes, QTBT I and QTBT II, are implemented for QTBT:

QTBT I: QTBT surfaces are rendered in such a way that the normal of a tessellation vertex is evaluated using (24) while the normal for each pixel is estimated by linear interpolation from the tessellation vertices.

QTBT II: QTBT surfaces are rendered with the normal of each pixel calculated by (24). It is more time-consuming than QTBT I.

Fig. 15 depicts the frames per second data by performing these approximations on five control meshes whose face numbers are, respectively, 12, 272, 1,284, 1,418, and 2,584. By changing the viewpoints, we obtain 12 groups of data in total. Each group includes four FPS numbers, which, respectively, correspond to QAS, QTBN, QTBT I, and QTBT II for the same model and the same viewpoint. The adaptive tessellation strategy is enabled for all experimental data. We think that the difference between QAS and QTBT is acceptable for these examples as a trade-off to improve the rendering quality.

6.5 Comparison with Smooth Surface Construction

Compared with PN and QAS, our approach achieves the bonus of higher rendering quality by sacrificing running time. We also made a comparison between our approximation and the SP by Peters [24]. Fig. 16 shows that the results by both approaches have a very close quality. Numerical analysis for mean geometric error and mean normal error also validates the statement. On the other hand, the SP approach needs to refine the control mesh at least once. For some special case, two rounds of refinement are required (extraordinary vertices with even valence). Therefore, each control triangle may result in at most $3 \times 4 + 1 = 13$ Bézier patches while our approach yields only one patch. In addition, in the SP approach, coefficients of different Bézier patches depend on each other. Its complete GPU implementation is not a trivial task.

7 CONCLUSION

We propose an approach to approximate Loop subdivision surfaces patch by patch using quartic triangular Bézier patches. It approximates a subdivision patch in two phases:

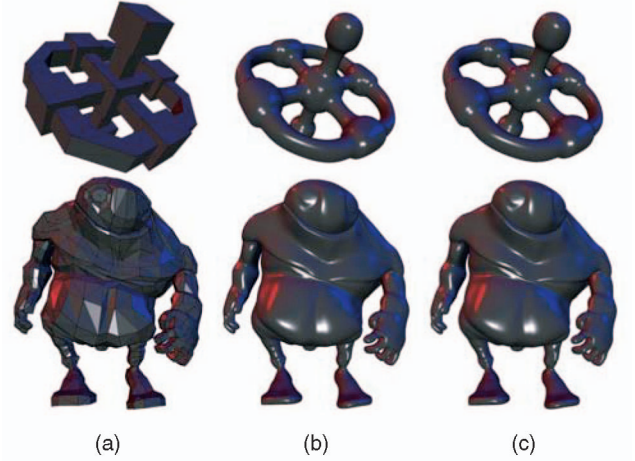


Fig. 16. Comparison between our approach and Peters' method. (a)-(c) Control meshes, QTBT, and SP, respectively.

interpolating the geometry of the subdivision patch first and then approximating the three parametric tangent fields of the patch. A blending strategy is introduced to address normal continuity.

The control coefficients of the Bézier patches may be explicitly expressed in terms of the vertices and their limit attributes on the local submeshes of the initial control mesh. The approach can exactly reproduce the subdivision path of a regular control triangle and faithfully reconstruct the regions for irregular triangles with high precision. Sharp and semisharp features can be achieved though no closed form is available in this case. We have also tried to reduce the complexity for generating a smooth normal field by directly interpolating the 15 limit normals on each patch. Finally, we have applied the idea to the approximation of ternary interpolatory subdivisions for textile-aided design [36].

As future work, it is useful to extend the approach to rendering of tri-quad subdivisions, composite subdivisions, and variants of subdivision surfaces such as displaced meshes, normal meshes, and multiresolution meshes. The algorithm can be further improved in the future to fit multiple Bézier surface patches at an extraordinary corner position in a single step with continuity conditions between these surface patches themselves and their neighboring surface patches, a kind of hole-filling algorithm. Finally, many applications based on subdivision surfaces, such as deformation, editing, Boolean operations, etc., may also benefit from the Bézier approximation and subsequent tessellation.

APPENDIX A

CONSTRUCTION OF MATRICES FOR SECTIONS 3 AND 4

As all rows for these matrices are sparse, we only give nonzero entries in the table format (column, weight). For instance, in Table 1, row = 0 and (column, weight) = (0, $5 - 8\alpha_{n_0}$) indicate that $8M_1(0, 0) = 5 - 8\alpha_{n_0}$.

TABLE 1
Matrix M_1 : Entries of $8M_1$ Are Listed

row	(column, weight)		(column, weight)	row	(column, weight)
0	$(0, 5 - 8\alpha_{n_0}), (18, 3 + 8\alpha_{n_0})$	7	$(0,3), (1, 1), (4, 1), (5, 3)$	14	$(2, 3), (13, 1), (14, 3), (15, 1)$
1	$(0,3), (1,3), (2, 1), (5, 1)$	8	$(0, 1), (1, 3), (5, 3), (6, 1)$	15	$(0, 1), (2, 3), (14, 1), (15, 3)$
2	$(0,3), (1,1), (2,3), (15, 1)$	9	$(1, 3), (5, 1), (6, 3), (7, 1)$	16	$(0, 3), (2, 1), (15, 3), (16, 1)$
3	$(1, 5 - 8\alpha_{n_1}), (19, 3 + 8\alpha_{n_1})$	10	$(1,3), (8,1), (9,3), (10, 1)$	17	$(0, 3), (15, 1), (16, 3), (17, 1)$
4	$(0,1), (1,3), (2,3), (10,1)$	11	$(1,3), (2,1), (9, 1), (10,3)$	18	$(18, 8)$
5	$(2, 5 - 8\alpha_{n_2}), (20, 3 + 8\alpha_{n_2})$	12	$(1,1), (2,3), (10,3), (11,1)$	19	$(19,8)$
6	$(0,3), (3, 1), (4,3), (5, 1)$	13	$(2, 3), (10, 1), (11,3), (12, 1)$	20	$(20,8)$

APPENDIX B

CLOSED FORMS FOR CONTROL COEFFICIENTS OF GEOMETRIC TRIANGULAR BÉZIER PATCHES

Assume that valences of vertices 0, 1, 2 in Fig. 3a are n_0, n_1 , and n_2 , respectively, and

$$a = \frac{5}{8} - \left(\frac{3}{8} + \frac{2}{8} \cos \frac{2\pi}{n_0} \right)^2, b = \frac{5}{8} - \left(\frac{3}{8} + \frac{2}{8} \cos \frac{2\pi}{n_1} \right)^2,$$

$$c = \frac{5}{8} - \left(\frac{3}{8} + \frac{2}{8} \cos \frac{2\pi}{n_2} \right)^2.$$

We list weights associated with the mask of Fig. 3a for computing $\mathbf{b}_{ijk}$ as follows (note that form $\mathbf{b}_{ijk} - \mathbf{v}$ indicates $\mathbf{b}_{ijk} = \mathbf{vP}$):

$$\mathbf{b}_{400} = [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0];$$

$$\mathbf{b}_{310} = \left[\frac{575}{576} - \frac{23}{18}a + \frac{1}{3}a^2, \frac{55}{192} - \frac{5}{18}b + \frac{1}{9}b^2, \frac{5}{36}, \frac{1}{192}, \frac{5}{96}, \frac{5}{36}, \right.$$

$$\left. \frac{1}{288}, \frac{1}{576}, 0, \frac{1}{576}, \frac{1}{288}, 0, 0, 0, 0, \frac{5}{96}, \frac{1}{192}, 0, \right.$$

$$\left. -\tau \frac{107}{192} + \frac{23}{18}a - \frac{1}{3}a^2, -\frac{25}{192} + \frac{5}{18}b - \frac{1}{9}b^2, 0 \right];$$

$$\mathbf{b}_{301} = \left[\frac{575}{576} - \frac{23}{18}a + \frac{1}{3}a^2, \frac{5}{36}, \frac{55}{192} - \frac{5}{18}c + \frac{1}{9}c^2, 0, \frac{1}{192}, \frac{5}{96}, 0, \right.$$

$$\left. 0, 0, 0, \frac{1}{288}, \frac{1}{576}, 0, \frac{1}{576}, \frac{1}{288}, \frac{5}{36}, \frac{5}{96}, \frac{1}{192}, -\frac{107}{192} + \frac{23}{18}a \right.$$

$$\left. -\frac{1}{3}a^2, 0, -\frac{25}{192} + \frac{5}{18}c - \frac{1}{9}c^2 \right];$$

TABLE 2
Matrix M_2 : Entries of $16M_2$ Are Listed

row	(column, weight)	row	(column, weight)
0	$(0, 10 - 16\alpha_{n_0}), (18, 6 + 16\alpha_{n_0})$	15	$(0,6), (1,2), (6,2), (7,6)$
1	$(0,6), (1,6), (2,2), (7,2)$	16	$(0,2), (1,6), (7,6), (8,2)$
2	$(0,6), (1,2), (2,6), (16,2)$	17	$(1,6), (3,2), (7,2), (8,6)$
3	$(0,1), (1,10), (2,1), (3,1), (4,1), (7,1), (8,1)$	18	$(1,2), (3,6), (8,6), (9,2)$
4	$(0,2), (1,6), (2,6), (4,2)$	19	$(3,6), (4,2), (10,2), (11,6)$
5	$(0,1), (1,1), (2,10), (4,1), (5,1), (15,1), (16,1)$	20	$(3,2), (4,6), (11,6), (12,6)$
6	$(1,6), (3,6), (4,2), (8,2)$	21	$(4,6), (5,2), (11,2), (12,6)$
7	$(1,6), (2,2), (3,2), (4,6)$	22	$(4,2), (5,6), (12,6), (13,2)$
8	$(1,2), (2,6), (4,6), (5,2)$	23	$(2,2), (5,6), (14,2), (15,6)$
9	$(2,6), (4,2), (5,6), (15,2)$	24	$(2,6), (5,2), (15,6), (16,2)$
10	$(3, 10 - 16\alpha_{n_1}), (19, 6 + 16\alpha_{n_1})$	25	$(0,2), (2,6), (15,2), (16,6)$
11	$(1,2), (3,6), (4,6), (11,2)$	26	$(0,6), (2,2), (16,6), (17,2)$
12	$(1,1), (2,1), (3,1), (4,10), (5,1), (11,1), (12,1)$	27	$(18,16)$
13	$(2,2), (4,6), (5,6), (12,2)$	28	$(19,16)$
14	$(5, 10 - 16\alpha_{n_2}), (20, 6 + 16\alpha_{n_2})$	29	$(20,16)$

TABLE 3
Matrix M_∞ : Entries of $12M_\infty$ Are Listed

row	(column, weight)	row	(column, weight)
0	(27,12)	8	(4,1), (5,1),(7,1),(8,6),(9,1),(12,1),(13,1)
1	(0,1), (1,6),(2,1),(3,1),(4,1),(15,1),(16,1)	9	(5,1), (8,1),(9,6),(13,1),(14,1),(23,1),(24,1)
2	(0,1), (1,1),(2,6),(4,1),(5,1),(25,1),(26,1)	10	(28,12)
3	(1,1), (3,6),(4,1),(6,1),(7,1),(16,1),(17,1)	11	(6,1), (7,1),(10,1),(11,6),(12,1),(19,1),(20,1)
4	(1,1), (2,1),(3,1),(4,6),(5,1),(7,1),(8,1)	12	(7,1), (8,1),(11,1),(12,6),(13,1),(20,1),(21,1)
5	(2,1), (4,1),(5,6),(8,1),(9,1),(24,1),(25,1)	13	(8,1), (9,1),(12,1),(13,6),(14,1),(21,1),(22,1)
6	(3,1), (6,6),(7,1),(10,1),(11,1),(17,1),(18,1)	14	(29,12)
7	(3,1), (4,1),(6,1),(7,6),(8,1),(11,1),(12,1)		

TABLE 4
Matrix M_R : Entries of $3M_R$ Are Listed

row	(column, weight)	row	(column, weight)
0	*	8	(4,-2), (5,2),(7,-4),(9,4),(12,-2),(13,2)
1	(2,4), (0,2),(4,2),(3,-2),(15,-2),(16,-4)	9	(5,-2), (8,-4),(13,-2),(14,2),(23,4),(24,2)
2	(0,-2), (1,-4),(4,-2),(5,2),(25,4),(26,2)	10	*
3	(1,2),(4,4),(6,-2),(7,2),(16,-2),(17,-4)	11	(6,-2), (7,2),(10,-4),(12,4),(19,-2),(20,2)
4	(1,-2), (2,2),(3,-4),(5,4),(7,-2),(8,2)	12	(7,-2), (8,2),(11,-4),(13,4),(20,-2),(21,2)
5	(2,-2), (4,-4),(8,-2),(9,2),(24,4),(25,2)	13	(8,-2), (9,2),(12,-4),(14,4),(21,-2),(22,2)
6	(3,2),(7,4),(10,-2),(11,2),(17,-2),(18,-4)	14	*
7	(3,-2), (4,2),(6,-4),(8,4),(11,-2),(12,2)		

$$\mathbf{b}_{220} - \left[\frac{1}{216} + \frac{22}{27}a - \frac{8}{27}a^2, \frac{1}{216} + \frac{22}{27}b - \frac{8}{27}b^2, \frac{4}{27}, -\frac{1}{216}, \right. \\ \left. -\frac{1}{108}, \frac{4}{27}, -\frac{1}{108}, -\frac{1}{216}, 0, -\frac{1}{216}, -\frac{1}{108}, 0, 0, 0, 0, \right. \\ \left. -\frac{1}{108}, -\frac{1}{216}, 0, \frac{3}{8} - \frac{22}{27}a + \frac{8}{27}a^2, \frac{3}{8} - \frac{22}{27}b + \frac{8}{27}b^2, 0 \right];$$

$$\mathbf{b}_{130} - \left[\frac{55}{192} - \frac{5}{18}a + \frac{1}{9}a^2, \frac{575}{576} - \frac{23}{18}b + \frac{1}{3}b^2, \frac{5}{36}, \frac{1}{576}, \frac{1}{288}, \frac{5}{36}, \right. \\ \left. \frac{5}{96}, \frac{1}{192}, 0, \frac{1}{192}, \frac{5}{96}, 0, 0, 0, 0, \frac{1}{288}, \frac{1}{576}, 0, -\frac{25}{192} + \frac{5}{18}a \right. \\ \left. - \frac{1}{9}a^2, -\frac{107}{192} + \frac{23}{18}b - \frac{1}{3}b^2, 0 \right];$$

$$\mathbf{b}_{211} - \left[\frac{11}{96} + \frac{49}{108}a - \frac{8}{27}a^2, \frac{581}{1728} - \frac{7}{36}b + \frac{1}{9}b^2, \frac{581}{1278} - \frac{7}{36}c \right. \\ \left. + \frac{1}{9}c^2, -\frac{1}{432}, -\frac{17}{576}, \frac{1}{54}, \frac{7}{864}, \frac{1}{864}, \frac{1}{1728}, \frac{1}{1728}, \frac{13}{288}, \frac{5}{1728}, \right. \\ \left. \frac{1}{1728}, \frac{1}{864}, \frac{7}{864}, \frac{1}{54}, -\frac{17}{576}, -\frac{1}{432}, \frac{49}{96} - \frac{49}{108}a + \frac{8}{27}a^2, \right. \\ \left. -\frac{31}{192} + \frac{7}{36}b - \frac{1}{9}b^2, -\frac{31}{192} + \frac{7}{36}c - \frac{1}{9}c^2 \right];$$

$$\mathbf{b}_{121} - \left[\frac{581}{1728} - \frac{7}{36}a + \frac{1}{9}a^2, \frac{11}{96} + \frac{49}{108}b - \frac{8}{27}b^2, \frac{581}{1728} - \frac{7}{36}c \right. \\ \left. + \frac{1}{9}c^2, \frac{1}{864}, \frac{7}{864}, \frac{1}{54}, -\frac{17}{576}, -\frac{1}{432}, -\frac{1}{432}, -\frac{17}{576}, \frac{1}{54}, \frac{7}{864}, \right. \\ \left. \frac{1}{864}, \frac{1}{1728}, \frac{13}{1728}, \frac{5}{288}, \frac{13}{1728}, \frac{1}{1728}, -\frac{31}{192} + \frac{7}{36}a - \frac{1}{9}a^2, \frac{49}{96} \right. \\ \left. - \frac{49}{108}b + \frac{8}{27}b^2, -\frac{31}{192} + \frac{7}{36}c - \frac{1}{9}c^2 \right];$$

$$\mathbf{b}_{202} - \left[\frac{1}{216} + \frac{22}{27}a - \frac{8}{27}a^2, \frac{4}{27}, \frac{1}{216} + \frac{22}{27}c - \frac{8}{27}c^2, 0, -\frac{1}{216}, \right. \\ \left. -\frac{1}{108}, 0, 0, 0, 0, -\frac{1}{108}, -\frac{1}{216}, 0, -\frac{1}{216}, -\frac{1}{108}, \frac{4}{27}, -\frac{1}{108}, \right. \\ \left. -\frac{1}{216}, \frac{3}{8} - \frac{22}{27}a + \frac{8}{27}a^2, 0, \frac{3}{8} - \frac{22}{27}c + \frac{8}{27}c^2 \right];$$

$$\mathbf{b}_{112} - \left[\frac{581}{1728} - \frac{7}{36}a + \frac{1}{9}a^2, \frac{581}{1728} - \frac{7}{36}b + \frac{1}{9}b^2, \frac{11}{96} + \frac{49}{108}c \right. \\ \left. - \frac{8}{27}c^2, \frac{1}{1728}, \frac{13}{1728}, \frac{5}{288}, \frac{13}{1728}, \frac{1}{1728}, \frac{1}{864}, \frac{7}{864}, \frac{1}{54}, -\frac{17}{576}, \right. \\ \left. -\frac{1}{432}, -\frac{1}{432}, -\frac{17}{576}, \frac{1}{54}, \frac{7}{864}, \frac{1}{864}, -\frac{31}{192} + \frac{7}{36}a - \frac{1}{9}a^2, \right. \\ \left. -\frac{31}{192} + \frac{7}{36}b - \frac{1}{9}b^2, \frac{49}{96} - \frac{49}{108}c + \frac{8}{27}c^2 \right];$$

APPENDIX D

PROOF OF REMARK 1

Proof. Given a cubic triangular Bézier patch $R_3(r, s, t)$, let $R_4(r, s, t)$ be the quartic triangular Bézier patch interpolating the 15 uniformly sampled points on $R_3(r, s, t)$ and $R_4^E(r, s, t)$ the elevation of $R_3(r, s, t)$. We then have

$$R_4(r_i, s_j, t_k) = R_4^E(r_i, s_j, t_k), \text{ for } r_i = 2^{i-4}, s_j = 2^{j-4}, \\ t_k = 1 - r_i - s_j, (0 \leq i \leq 4, 0 \leq j \leq 4 - i, k = 1 - i - j).$$

Denote the coefficients vector of $R_4(r, s, t)$ and $R_4^E(r, s, t)$ by $\mathbf{R}$ and $\mathbf{R}^E$, respectively. It follows that

$$\mathbf{M}_B(\mathbf{R} - \mathbf{R}^E) = \mathbf{0},$$

where $\mathbf{M}_B$ is the matrix defined in (10). It yields

$$\mathbf{R} - \mathbf{R}^E = \mathbf{0},$$

and therefore, $R_4(r, s, t) = R_4^E(r, s, t)$ holds. $\square$

ACKNOWLEDGMENTS

This paper is supported by the Natural Science Foundation of China under grant no. 60673005, City University of Hong Kong under SRG grant no. 7001928, the Research Grants Council of Hong Kong SAR under CERG grant no. CityU 1184/06E, the Guangdong Natural Science Foundation under grant no. 9151064101000106, and the Open Project from the State Key Lab of CAD&CG, Zhejiang University. The authors would also like to thank AIM@SHAPE Shape Repository for the Fandisk and Mechanical part models, and Princeton Shape Benchmark for some of the other models.

REFERENCES

- [1] S. Bischoff, L. Kobbelt, and H.-P. Seidel, "Towards Hardware Implementation of Loop Subdivision," *Proc. Eurographics/SIGGRAPH Workshop Graphics Hardware*, pp. 41-50, 2000.
- [2] J. Stam, "Exact Evaluation of Catmull-Clark Subdivision Surfaces at Arbitrary Parameter Values," *Proc. ACM SIGGRAPH*, pp. 395-400, 1998.
- [3] T. Boubekeur and C. Schlick, "Approximation of Subdivision Surfaces for Interactive Applications," *Proc. ACM SIGGRAPH*, 2007.
- [4] T. Boubekeur and C. Schlick, "QAS: Real-Time Quadratic Approximation of Subdivision Surfaces," *Proc. 15th Pacific Conf. Computer Graphics and Applications*, pp. 453-456, 2007.
- [5] C. Loop and S. Schaefer, "Approximating Catmull-Clark Subdivision Surfaces with Bicubic Patches," *ACM Trans. Graphics*, vol. 27, no. 1, pp. 8:1-8:11, 2008.
- [6] K. Pulli and M. Segal, "Fast Rendering of Subdivision Surfaces," *Proc. Seventh Eurographics Workshop Rendering*, pp. 61-70, 1996.
- [7] J. Bolz and P. Schröder, "Rapid Evaluation of Catmull-Clark Subdivision Surfaces," *Proc. Seventh Int'l Conf. Web3D Symp.*, pp. 11-18, 2002.
- [8] Y. Yasui and T. Kanai, "Surface Quality Assessment of Subdivision Surfaces on Programmable Graphics Hardware," *Proc. Int'l Conf. Shape Modeling*, pp. 129-138, 2004.
- [9] T. Kanai and Y. Yasui, "Per-Pixel Evaluation of Parametric Surfaces on GPU," *Proc. ACM Workshop General Purpose Computing on Graphics Processors*, pp. 113-120, 2004.
- [10] J. Fung, "Towards Adaptive Rendering of Smooth Primitives on GPUs," master's thesis, Univ. of British Columbia, 2005.
- [11] L.-J. Shiue, V. Goel, and J. Peters, "Mesh Mutation in Programmable Graphics Hardware," *Proc. SIGGRAPH/Eurographics Workshop Graphics Hardware*, pp. 15-24, 2003.
- [12] L.-J. Shiue, I. Jones, and J. Peters, "A Realtime GPU Subdivision Kernel," *ACM Trans. Graphics*, vol. 22, no. 3, pp. 1010-1015, 2005.
- [13] M. Kim and J. Peters, "Realtime Loop Subdivision on the GPU," *Proc. ACM SIGGRAPH*, 2005.
- [14] A. Vlachos, J. Peters, C. Boyd, and J.L. Mitchell, "Curved PN Triangles," *Proc. 2001 Symp. Interactive 3D Graphics*, pp. 159-166, 2001.
- [15] K. Chung and L.-S. Kim, "A PN Triangle Generation Unit for Fast and Simple Tessellation Hardware," *Proc. Int'l Symp. Circuits and System*, pp. 728-731, 2003.
- [16] C. Fünfzig, K. Müller, D. Hansford, and G. Farin, "PNG1 Triangles for Tangent Plane Continuous Surfaces on the GPU," *Proc. Conf. Graphics Interface*, pp. 219-226, 2008.
- [17] T. Boubekeur and C. Schlick, "Generic Mesh Refinement on GPU," *Proc. SIGGRAPH/Eurographics Workshop Graphics Hardware*, pp. 99-104, 2005.
- [18] T. Boubekeur and C. Schlick, "A Flexible Kernel for Adaptive Mesh Refinement on GPU," *Computer Graphics Forum*, vol. 27, no. 2, pp. 102-113, 2008.
- [19] M. Kazakov, "Catmull-Clark Subdivision for Geometry Shaders," *Proc. Fifth Int'l Conf. Computer Graphics, Virtual Reality, Visualisation and Interaction in Africa*, pp. 77-84, 2007.
- [20] T. Boubekeur and M. Alexa, "Phong Tessellation," *ACM Trans. Graphics*, vol. 27, no. 5, pp. 141:1-141:5, 2008.
- [21] M. Alexa and T. Boubekeur, "Subdivision Shading," *ACM Trans. Graphics*, vol. 27, no. 5, pp. 142-145, 2008.
- [22] I. Castaño, "Next-Generation Subdivision Surfaces," Siggraph talk, <http://developer.nvidia.com/object/siggraph-2008-Subdiv.html>, 2008.
- [23] D. Kovacs, J. Mitchell, S. Drone, and D. Zorin, "Real-Time Creased Approximate Subdivision Surfaces," *Proc. Symp. Interactive 3D Graphics and Games*, pp. 155-160, 2009.
- [24] J. Peters, "Smooth Patching of Refined Triangulations," *ACM Trans. Graphics*, vol. 20, no. 1, pp. 1-9, 2001.
- [25] C. Loop, "Generalized B-Spline Surfaces of Arbitrary Topological Type," PhD dissertation, Dept. of Computer Science and Eng., Univ. of Washington, 1992.
- [26] C. Loop, "Smooth Subdivision Surfaces Based on Triangles," master's thesis, Dept. of Math., Univ. of Utah, 1987.
- [27] B. Dudash, "Tessellation of Displaced Subdivision Surfaces in DX11," GPU-BBQ, <http://developer.nvidia.com/object/gpubbq-2008-subdiv.html>, 2008.
- [28] H. Hoppe, T. DeRose, T. Duchamp, M. Halstead, H. Jin, J. McDonald, J. Schweitzer, and W. Stuetzle, "Piecewise Smooth Surface Construction," *Proc. ACM SIGGRAPH*, pp. 295-302, 1994.
- [29] L. Barthe and L. Kobbelt, "Direct Computation of a Control Vertex Position on Any Subdivision Level," *Mathematics of Surfaces*, pp. 40-47, Springer-Verlag, 2003.
- [30] J. Lai, "Fortran Subroutines for B-Nets of Box Splines on Three- and Four-Directional Meshes," *Numerical Algorithms*, vol. 2, pp. 33-38, 1992.
- [31] J. Stam, "Evaluation of Loop Subdivision Surfaces," *Proc. ACM SIGGRAPH CDROM*, 1998.
- [32] C. Van Overveld and B. Wyvill, "Phong Normal Interpolation Revisited," *ACM Trans. Graphics*, vol. 16, no. 5, pp. 397-419, 1997.
- [33] K. Gee, "Direct3D 11 Tessellation," <http://www.microsoft.com/downloads/Direct3D11Tessellation.rar>, 2010.
- [34] A. Klein, "Introduction to the Direct3D 11 Graphics Pipeline," [http://www.microsoft.com/downloads/Introduction to the Direct3D11GraphicsPipeline.rar](http://www.microsoft.com/downloads/Introduction%20to%20the%20Direct3D11GraphicsPipeline.rar), 2010.
- [35] A. Dyken, M. Reimers, and J. Seland, "Semi-Uniform Adaptive Patch Tessellation," *Computer Graphics Forum*, vol. 28, no. 8, pp. 2255-2263, 2009.
- [36] J. Zhang, D. Zheng, C. Liang, G. Li, G. Baciú, and J. Hu, "TTDSS Model and Tessellation of Woven Fabrics for the Next Generation GPUs," Mar. 2009.



of CAD&CG, Zhejiang University.

Guiqing Li received the BSc degree from the University of Science and Technology of China in 1987, the MSc degree from Nankai University, Tianjin, in 1990, and the PhD degree from the Institute of Computing Technology, Beijing, in 2001. He is a professor in the College of Computer Science and Engineering, South China University of Technology (SCUT). Before joining SCUT in September 2003, he worked as a postdoctoral researcher in the State Key Lab



Jiahua Zhang received the BSc and MSc degrees from South China University of Technology in 2007 and 2009, respectively. He is currently working toward the PhD degree at the Hong Kong Polytechnic University.



Canjiang Ren received the BSc degree from East China University of Technology in 2006 and the MSc degree from South China University of Technology in 2009.



Weiyan Ma received the BSc and MSc degrees from East China Institute of Technology (ECIT) in 1982 and 1985, respectively, and the MEng and PhD degrees from Katholieke Universiteit Leuven (K.U.Leuven) in 1989 and 1994, respectively. He is an associate professor of mechanical engineering at City University of Hong Kong (CityU). He lectures in the areas of geometric modeling, CAD/CAM, and rapid prototyping. Prior to joining CityU, he worked as a research fellow at Materialise N.V., a rapid prototyping firm in Belgium. He also worked in the Department of Mechanical Engineering at K.U.Leuven, Belgium, and the Department of Space Vehicle Engineering at ECIT, Nanjing, China. His present research interests include CAGD, CAD/CAM, virtual design and manufacturing, rapid prototyping, and reverse engineering.

► **For more information on this or any other computing topic, please visit our Digital Library at www.computer.org/publications/dlib.**